Prompt 1A — Abstracción de Entidades UML

Sistema EcoHydro-4 — IC-2101 Programación Orientada a Objetos, ITCR

---

1. UnidadCentralControl

Justificación

Es la clase central del dominio: coordina el brazo robótico, la bitácora y el operador. Concentra las reglas de negocio RF1 (diagnóstico) y RF2 (muestreo). Sin ella no existe el sistema; ninguna otra clase puede asumir su rol sin violar el principio de responsabilidad única.

Atributos privados

Visibilidad	Tipo Java	Nombre
private	String	ubicacion
private	String	codigoInterno
private	String	estadoFuncionamiento
private	double	limitePotenciaW
private	BrazoRobotico	brazoRobotico
private	BitacoraHistorica	bitacora
private	Operador	operador
private	boolean	compensacionTermicaActiva

Constantes de clase

Visibilidad	Tipo Java	Nombre	Valor
public static final	String	ESTADO_ACTIVO	"Activo"
public static final	String	ESTADO_BAJO_CONSUMO	"Bajo Consumo / Mantenimiento Requerido"
public static final	String	ESTADO_FALLA	"Falla Crítica"
public static final	String	ESTADO_FALLA_PERM	"Falla Crítica Permanente"
private static final	double	UMBRAL_HORAS_USO	1000.0
private static final	double	POTENCIA_AUXILIARES	50.0
private static final	double	FACTOR_REDUCCION	0.40
private static final	double	FACTOR_COMPENSACION_TERMICA	1.05

Métodos principales

```
// Constructor
public UnidadCentralControl(String ubicacion, String codigoInterno,
    double limitePotenciaW, double velBrazo, double anguloBrazo,
    BitacoraHistorica bitacora, Operador operador)

// RF1 — Diagnóstico de viabilidad
public String diagnosticarViabilidad()

// RF2 — Ciclo de muestreo (Dependencia UML: ReporteMuestreo como variable local)
public ReporteMuestreo ejecutarCicloMuestreo(double ph,
    double conductividad, double tiempoPrueba)
```

```
// RF4 — Integración climática
public void aplicarClima(ConsultaClimatica clima)
public boolean isCompensacionTermicaActiva()

// RF6/2G — Destrucción (demo de Composición)
public void destruir()

// Getters
public String getUbicacion()
public String getCodigoInterno()
public String getEstadoFuncionamiento()
public double getLimitePotenciaW()
public BrazoRobotico getBrazoRobotico()
public BitacoraHistorica getBitacora()
public Operador getOperador()

// Setters validados
public void setUbicacion(String ubicacion)
public void setCodigoInterno(String codigoInterno)
public void setEstadoFuncionamiento(String estado)
public void setLimitePotenciaW(double limitePotenciaW)
public void setBitacora(BitacoraHistorica bitacora)
public void setOperador(Operador operador)

// toString
public String toString()
```

---

2. BrazoRobotico

Justificación

Modela el actuador físico del sistema hidropónico. Encapsula sus propiedades mecánicas (velocidad, ángulo, horas de uso) y las expone con validación. Es responsabilidad exclusiva de esta clase mantener la coherencia de sus parámetros cinemáticos; delegar esa lógica en `UnidadCentralControl` violaría el principio de responsabilidad única. Su existencia está ligada exclusivamente a la unidad (Composición).

Atributos privados

Visibilidad	Tipo Java	Nombre
private	double	velocidadDesplazamiento
private	double	anguloExtension
private	double	horasUsoAcumuladas

Métodos principales

```
// Constructor
public BrazoRobotico(double velocidadDesplazamiento,
    double anguloExtension, double horasUsoAcumuladas)

// Getters
public double getVelocidadDesplazamiento()
public double getAnguloExtension()
public double getHorasUsoAcumuladas()

// Setters validados
public void setVelocidadDesplazamiento(double velocidadDesplazamiento) // > 0
public void setAnguloExtension(double anguloExtension)                 // 0–180°
public void setHorasUsoAcumuladas(double horasUsoAcumuladas)           // >= 0

// toString
public String toString()
```

---

3. BitacoraHistorica

Justificación

Gestiona la colección persistente de reportes de muestreo y centraliza la lógica de análisis químico (RF3): detección de anomalías, cálculo de porcentajes y generación de resúmenes. Separar estas responsabilidades de `UnidadCentralControl` evita que esa clase crezca indefinidamente (God Object). La bitácora sobrevive a la unidad (Agregación), lo que refuerza su independencia como clase propia.

Atributos privados

Visibilidad	Tipo Java	Nombre
private final	String	id
private final	ArrayList<ReporteMuestreo>	reportes

Constantes de clase

Visibilidad	Tipo Java	Nombre	Valor
public static final	double	PH_MIN	5.5
public static final	double	PH_MAX	6.5
public static final	double	COND_MIN	1.2
public static final	double	COND_MAX	2.4

Métodos principales

```
// Constructor
public BitacoraHistorica(String id)

// RF3.1 — Registro
public void registrarReporte(ReporteMuestreo reporte)

// RF3.2 — Detección de anomalías
public boolean esAnomaliaQuimica(ReporteMuestreo r)
public String describirAnomalia(ReporteMuestreo r)

// RF3.3 — Estadísticas
public double calcularPorcentajeAnomalias()
public int getTotalReportes()

// RF3.4 — Resumen completo
public String generarResumen()

// Getters
public String getId()
public ArrayList<ReporteMuestreo> getReportes()

// toString
public String toString()
```

---

4. ReporteMuestreo

Justificación

Representa el resultado inmutable de un ciclo de medición hidropónica. Agrupa cuatro valores que siempre viajan juntos (fecha-hora, pH, conductividad, gasto energético) y que pierden significado si se separan. Es una clase de valor (*value object*): se crea en un instante, no se modifica y su ciclo de vida es independiente del brazo que lo generó (Dependencia UML). Tener sus propios getters y `toString` evita que `BitacoraHistorica` o `UnidadCentralControl` accedan directamente a campos ajenos, respetando el encapsulamiento.

Atributos privados

Visibilidad	Tipo Java	Nombre
private	LocalDateTime	fechaHora

`private`	`double`	`ph`
`private`	`double`	`conductividadElectrica`
`private`	`double`	`gastoEnergeticoW`

Métodos principales

```java
// Constructor (validado: ph 0–14, conductividad >= 0, gasto >= 0)
public ReporteMuestreo(LocalDateTime fechaHora, double ph,
    double conductividadElectrica, double gastoEnergeticoW)

// Getters (sin setters: objeto inmutable por diseño)
public LocalDateTime getFechaHora()
public double getPh()
public double getConductividadElectrica()
public double getGastoEnergeticoW()

// toString
public String toString()
```

---

5. Operador

Justificación

Modela al ser humano responsable del sistema. Tiene identidad propia (ficha de empleado, nombre, correo) que existe antes y después de su asignación a una unidad hidropónica. Encapsula la validación de su correo electrónico (debe contener `@`) y de sus campos obligatorios. Es el destinatario de las alertas (RF5), lo que le da un rol funcional claro que justifica su existencia como clase independiente en lugar de una simple cadena de texto dentro de `UnidadCentralControl`.

Atributos privados

Visibilidad	Tipo Java	Nombre
`private`	`String`	`fichaEmpleado`
`private`	`String`	`nombreCompleto`
`private`	`String`	`correoElectronico`

Métodos principales

```java
// Constructor
public Operador(String fichaEmpleado, String nombreCompleto,
    String correoElectronico)

// Getters
public String getFichaEmpleado()
public String getNombreCompleto()
public String getCorreoElectronico()

// Setters validados
public void setFichaEmpleado(String fichaEmpleado)     // no nulo/vacío
public void setNombreCompleto(String nombreCompleto)   // no nulo/vacío
public void setCorreoElectronico(String correo)        // debe contener '@'

// toString
public String toString()
```
